

```
[3]: import numpy as np
import pandas as pd
```

```
[7]: myindex = ['USA','Canada','Mexico']
mydata = ['1776','1867','1821']
```

We see that the indices of elements in this array are normal integers

```
[13]: mySeries = pd.Series(data = mydata)
mySeries
```

```
[13]: 0    1776
1    1867
2    1821
dtype: str
```

Now, we assign the list 'myindex' as index to mySeries - pd.Series automatically converts list to series

```
[16]: mySeries = pd.Series(data = mydata, index = myindex)
mySeries
```

```
[16]: USA        1776
Canada    1867
Mexico    1821
dtype: str
```

Above, we can see that the indices of mySeries are elements of myindex

To get elements on this series:

```
[21]: mySeries[0]
```

```
-----
KeyError                                Traceback (most recent call last)
File ~/Machine Learning/Data Analysis/.venv/lib64/python3.14/site-packages/pandas/core/indexes/base.py:3641, in Index.get_loc(self, key)
    3640 try:
-> 3641     return self._engine.get_loc(casted_key)
    3642 except KeyError as err:

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()
--> 168 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/index.pyx:176, in pandas._libs.index.IndexEngine.get_loc()
--> 176 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/index.pyx:583, in pandas._libs.index.StringObjectEngine._check_type()
--> 583 'Could not get source, probably due dynamically evaluated source code.'

KeyError: 0

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call last)
Cell In[21], line 1
----> 1 mySeries[0]

File ~/Machine Learning/Data Analysis/.venv/lib64/python3.14/site-packages/pandas/core/series.py:959, in Series.__getitem__(self, key)
    954     key = unpack_ituple(key)
    956 elif key_is_scalar:
    957     # Note: GH#50617 in 3.0 we changed int key to always be treated as
    958     # a label, matching DataFrame behavior.
-> 959     return self._get_value(key)
    961 # Convert generator to list before going through hashable part
    962 # (We will iterate through the generator there to check for slices)
    963 if is_iterator(key):

File ~/Machine Learning/Data Analysis/.venv/lib64/python3.14/site-packages/pandas/core/series.py:1046, in Series._get_value(self, label, take_eable)
    1043     return self._values[label]
    1045 # Similar to Index.get value, but we do not fall back to positional
-> 1046 loc = self.index.get_loc(label)
    1048 if is_integer(loc):
    1049     return self._values[loc]

File ~/Machine Learning/Data Analysis/.venv/lib64/python3.14/site-packages/pandas/core/indexes/base.py:3648, in Index.get_loc(self, key)
    3643     if isinstance(casted_key, slice) or (
    3644         isinstance(casted_key, abc.Iterable)
    3645         and any(isinstance(x, slice) for x in casted_key)
    3646     ):
    3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
    3649 except TypeError:
    3650     # If we have a listlike key, _check_indexing_error will raise
    3651     # InvalidIndexError. Otherwise we fall through and re-raise
    3652     # the TypeError.
    3653     self._check_indexing_error(key)

KeyError: 0
```

In older versions of Pandas, this worked to get element at a particular position by numbers, now we've to use 'iloc'

```
[23]: print(mySeries.iloc[0])

1776
```

```
[24]: mySeries['USA']
```

```
[24]: '1776'
```

We can also create series from a dictionary

```
[25]: ages = {'Sam':5, 'Frank':10, 'Spike':7}
pd.Series(ages)
```

```
[25]: Sam        5
Frank       10
Spike        7
dtype: int64
```

```
[ ]:
```